

Chapitre 3 : Vues dynamiques en UML

1. Introduction

Nous avons mentionné dans le chapitre précédent qu'UML offre un ensemble de diagrammes qui peuvent être classés en deux catégories de vues : Les vues statiques et les vues dynamiques.

Pour les vues statiques, ils regroupent l'ensemble de diagrammes statiques ou se qu'on appelle les diagrammes de structure et qui montrent les aspects structurels d'un système et décrivent les composants qui constituent ce système et les relations entre eux.

Les vues dynamiques en UML représentent les aspects temporels et comportementaux d'un système logiciel. Elles offrent une perspective cruciale pour comprendre comment les objets interagissent et évoluent dans le temps au sein d'une application. Ces vues permettent de capturer les dynamiques du système, allant de la séquence des opérations à l'évolution des états des objets.

Les diagrammes dynamiques les plus important rassemblent :

- Le diagramme de séquence
- Le diagramme d'activités
- Le diagramme d'états-transitions
- Le diagramme de collaboration

2. Diagramme de séquence

Les diagrammes de séquence sont des outils graphiques qui illustrent la séquence d'interactions entre différents acteurs et objets d'un système au fil du temps. Ils mettent l'accent sur l'ordre chronologique des messages échangés entre les objets.

Les diagrammes de séquence sont utilisés pour formaliser le comportement du système et de visualiser la communication entre les objets. Ils sont utiles pour l'identification d'objets supplémentaires qui participent dans les cas d'utilisation. Nous appelons les objets impliqués dans un cas d'utilisation des objets participant. Un diagramme de séquence représente les interactions qui ont lieu entre ces objets.

2.1 Eléments principaux

Les éléments principaux d'un diagramme de séquence en UML comprennent les acteurs, les objets, les lignes de vie et les messages. En combinant ces éléments, un diagramme de séquence offre une représentation visuelle claire et détaillée des interactions entre les acteurs et les objets d'un système logiciel au fil du temps.

A. Les acteurs : Ils représentent des entités externes qui interagissent avec le système. Il peut s'agir d'utilisateurs, de systèmes externes, ou d'autres entités externes. Les acteurs sont généralement représentés par des rectangles lifelines verticaux, avec le nom de l'acteur écrit à l'intérieur du rectangle, ou le symbole  avec le nom de l'acteur écrit en bas du Symbole.

B. Les objets : Ils sont des instances de classes ou d'entités du système que l'on souhaite modéliser dans le diagramme de séquence. Chaque objet représente une occurrence spécifique d'une classe. Les objets sont généralement représentés par des rectangles ou des boîtes de vie situées sur la ligne de vie (lifeline) correspondant à l'instance de l'objet. Le nom de l'objet est

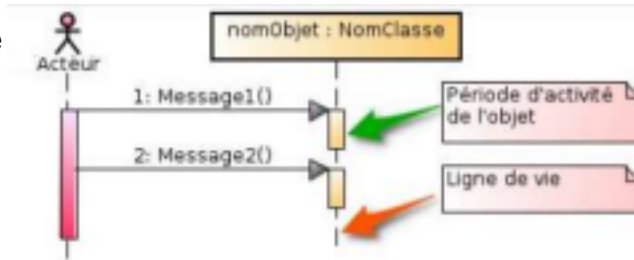
souvent inclus dans la boîte de vie.

1

C. Les lignes de Vie (Lifelines) : Elles représentent la durée de vie d'une instance d'un objet. Elles montrent quand un objet est actif et quand il ne l'est pas. Les lignes de vie sont des lignes verticales adjacentes à l'acteur ou à l'objet, indiquant le temps pendant lequel l'acteur ou l'objet est en vie et actif.

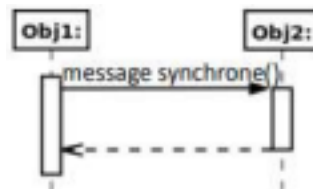
D. Les messages : Ils représentent les communications entre les acteurs et les objets. Ils décrivent les interactions qui se produisent au fil du temps et indiquent la séquence d'exécution des actions. Les messages sont représentés par des flèches entre les lifelines des acteurs et des objets. Ils sont généralement étiquetés avec le nom du message et, le cas échéant, avec des informations supplémentaires telles que les paramètres. Un message définit une communication particulière entre des lignes de vie et peut concerner l'envoi d'un signal, l'invocation d'une opération et la

la destruction d'une instance.



Il existe plusieurs types de Messages :

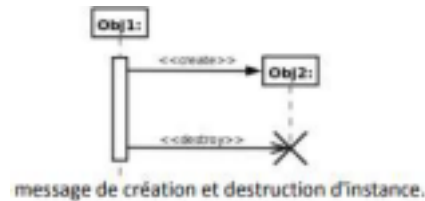
- **Message Synchrone** : Indique une interaction immédiate entre l'expéditeur et le destinataire, où l'expéditeur attend une réponse avant de poursuivre. L'invocation d'une opération est le type de message le plus utilisé en programmation objet. Graphiquement, un message synchrone se représente par une flèche en traits pleins et à l'extrémité pleine partant de la ligne de vie d'un objet expéditeur et allant vers celle de l'objet cible. Ce message peut être suivi d'une réponse qui se représente par une flèche en pointillé.



- **Message Asynchrone** : Indique une interaction où l'expéditeur n'attend pas de réponse immédiate et peut continuer son exécution. Une interruption ou un événement sont de bons exemples de signaux. Graphiquement, un message asynchrone se représente par une flèche en traits pleins et à l'extrémité ouverte partant de la ligne de vie d'un objet expéditeur et allant vers celle de l'objet cible.

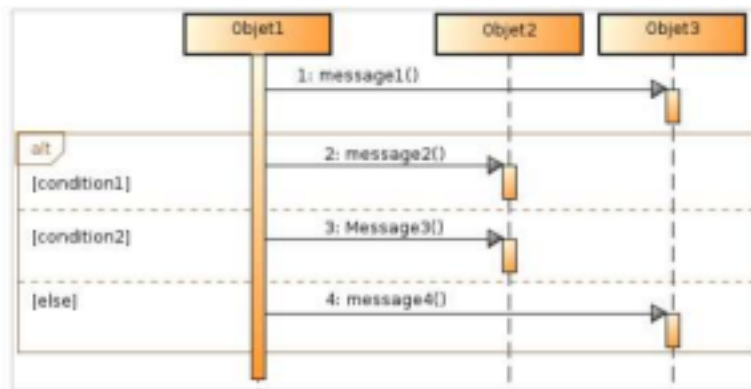


- **Messages de création et destruction d'instance** : La création d'un objet est matérialisée par une flèche qui pointe sur le sommet d'une ligne de vie. La destruction d'un objet est matérialisée par une croix qui marque la fin de la ligne de vie de l'objet. La destruction d'un objet n'est pas nécessairement consécutive à la réception d'un message.

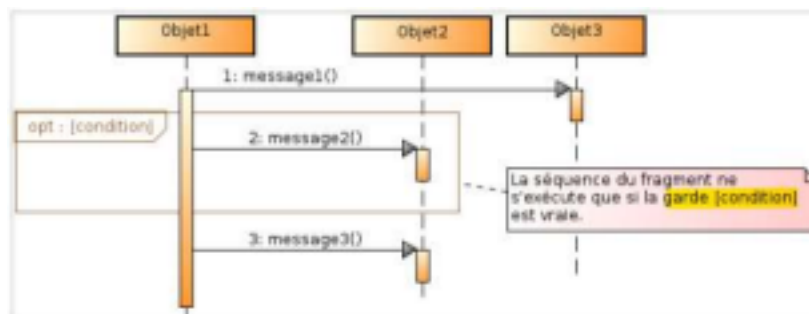


E. Les fragments d'interaction combinés : Ils permettent de modéliser des structures de contrôle telles que les boucles et les branchements conditionnels au sein d'un diagramme de séquence. Un fragment est représenté dans un rectangle dont le coin supérieur gauche contient un pentagone. Dans le pentagone figure le type de la combinaison, appelé opérateur d'interaction. Les opérateurs d'interaction les plus utilisés sont : alternative, optionel, parallèle et loop

- Alternative (alt) : L'opérateur alt est utilisé pour représenter des branches conditionnelles dans un diagramme de séquence. Il peut être utilisé pour modéliser des choix conditionnels où différentes séquences d'actions sont suivies en fonction de conditions spécifiques.



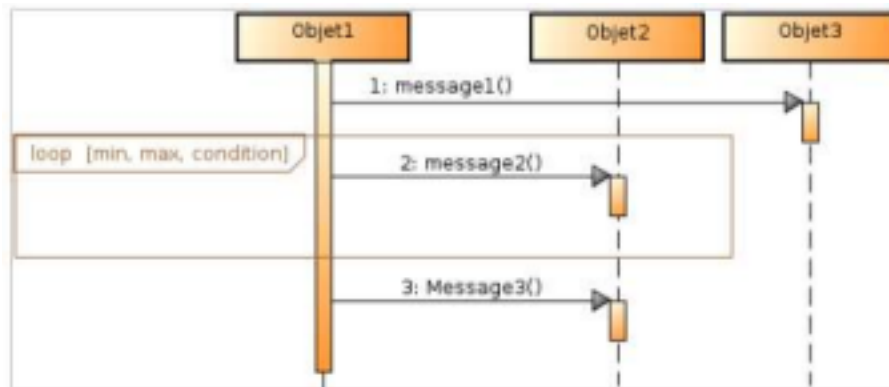
- Optionel (opt) : L'opérateur opt est utilisé pour représenter des éléments qui peuvent être présents ou non, en fonction d'une condition. Il peut être utile pour modéliser des scénarios où une séquence d'actions peut être exécutée en fonction de la satisfaction d'une condition, mais n'est pas obligatoire.



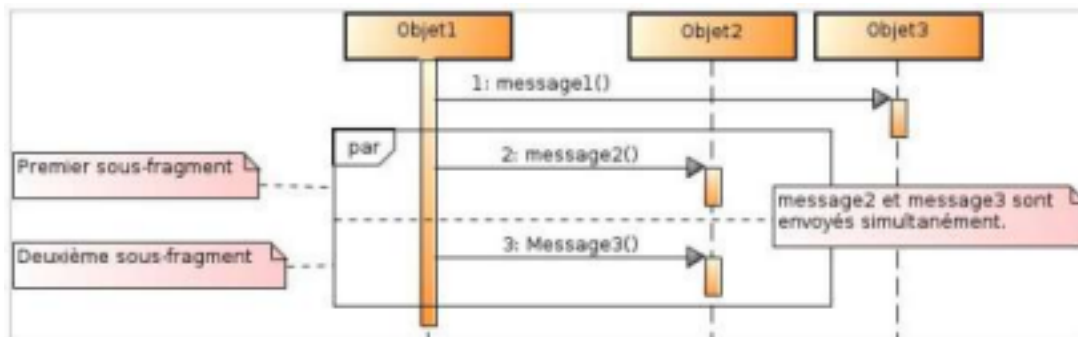
- Boucle (loop) : L'opérateur loop est utilisé pour représenter des boucles dans un diagramme de séquence. Il peut être employé pour modéliser des situations où une séquence d'actions doit être

répétée plusieurs fois en fonction d'une condition.

3



- Parallèle (par) : L'opérateur loop est utilisé pour représenter des éléments qui s'exécutent simultanément (traitements



concurrents
).

2.2 Exemple

Soit le diagramme de séquence pour le cas d'utilisation mise à jour de l'heure (MAJHeure) de notre simple montre, et qui

permet de modifier l'heure en lui ajoutant une minute.

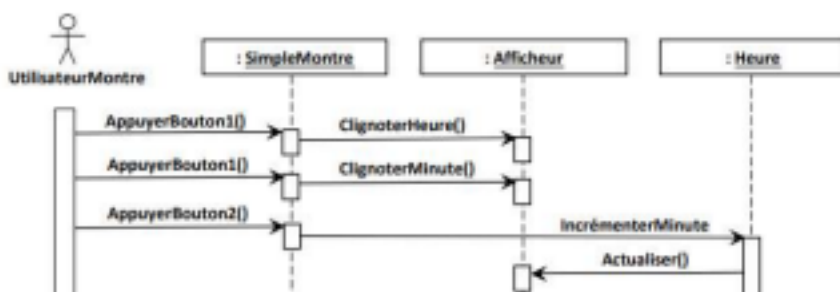


Diagramme de séquence de MAJHeure

4

La colonne de gauche représente le scénario de l'acteur UtilisateurMontre qui initie le cas d'utilisation. Les autres colonnes représentent la chronologie des objets qui participent à ce cas d'utilisation. Dans ce cas, l'UtilisateurMontre appuie deux fois sur le bouton et une fois sur le bouton 2 pour régler sa montre d'une minute d'avance. Le cas d'utilisation de MAJHeure se termine lorsque l'UtilisateurMontre appuie sur les deux boutons simultanément.

3. Diagramme d'états-transitions

Les diagrammes d'état-transition décrivent le comportement d'un objet individuel par un certain nombre d'états et de transitions entre ces états. Ils présentent les séquences possibles d'états et d'actions qu'une instance de classe peut traiter au cours de son cycle de vie en réaction à des événements.

Les diagrammes d'états-transitions sont un outil puissant pour modéliser le comportement dynamique des systèmes, offrant une vue intuitive du changement d'états en réponse à des événements spécifiques.

3.1 Eléments clés

A. État : Il représente une condition ou une situation spécifique dans le cycle de vie d'un objet ou d'un système. Il décrit une phase ou un mode dans lequel l'entité peut se trouver, caractérisée par un ensemble spécifique de propriétés, de comportements ou de valeurs des attributs.

B- Transition : Elle représente le passage d'un état à un autre dans un diagramme d'états en réponse à un événement spécifique. Elle indique un changement d'état déclenché par un événement particulier.

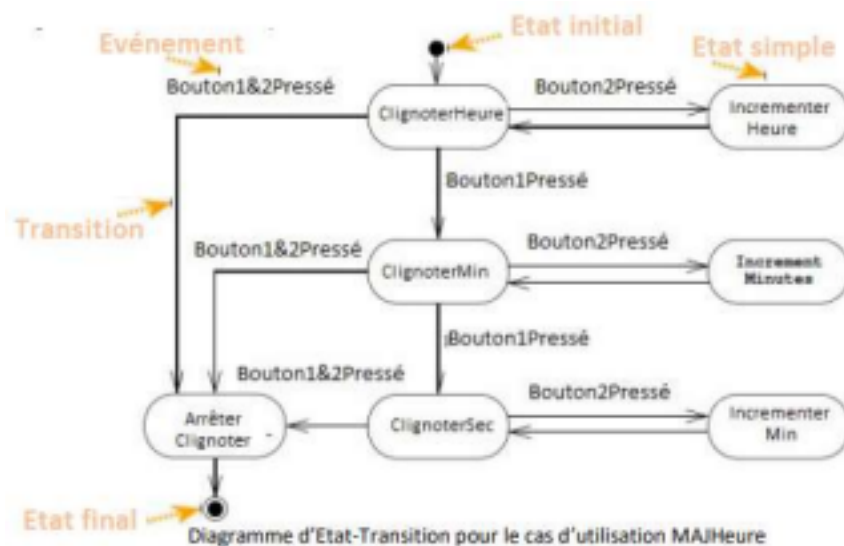
C- Événement : C'est un stimulus externe ou interne qui déclenche une transition entre les états d'un objet ou d'un système. Les événements sont des déclencheurs qui provoquent un changement d'état. Un événement se produit à un instant précis et est dépourvu de durée. Quand un événement est reçu, une transition peut être déclenchée et faire basculer l'objet dans un nouvel état.

On peut diviser les événements en plusieurs types explicites et implicites : signal, appel, changement et temporel.

1. Les événements de type signal est déclenché par La réception d'un signal qui est un événement pour l'objet destinataire. Un même objet peut être à la fois expéditeur et destinataire.
2. Les événements d'appel (call) sont les événements déclenchés par des méthodes déclarées au niveau du diagramme de classes.
3. Les événements de changement (change) sont d'une manière déclarative d'attendre qu'une condition soit satisfaite.
4. Les événements temporels (after ou when) Le temps commence à s'écouler dès l'entrée dans l'état courant.

3.2 Exemple illustrant la représentation Graphique

Soit le diagramme d'état-transition pour le cas MAJHeure :



Ce diagramme représente une information différente du diagramme de séquence précédant. Le diagramme de séquence se concentre sur les messages échangés entre les objets à la suite d'événements extérieurs créés par des acteurs. Le diagramme d'état-transition se concentre sur les transitions entre états à la suite des événements externes pour un objet individuel.

3.3 Point de choix

Il est possible de représenter des alternatives telles que les points de jonction (représentés par un petit cercle plein) et les points de décision (représentés par un losange).

A. Point de jonction :

- Le point de jonction représente le branchement d'une clause conditionnelle.
- Les points de jonction sont un artefact graphique (un pseudo état) qui permet de partager

des segments de transition.

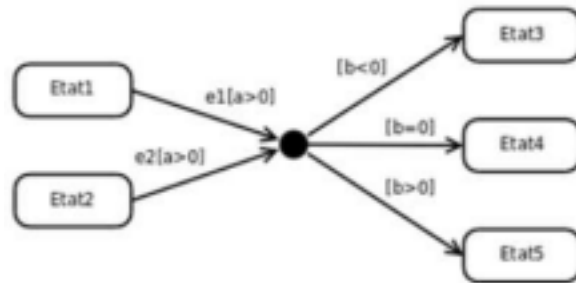
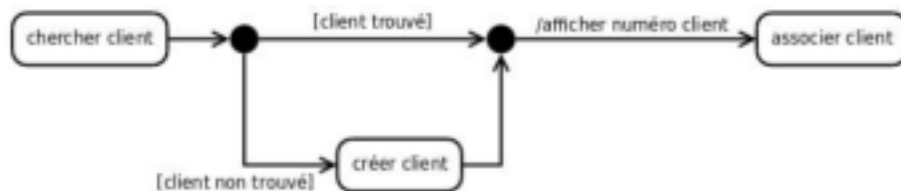


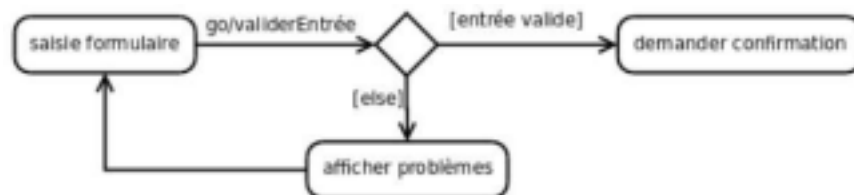
Diagramme représentant un point de jonction.



Exemple d'utilisation de deux points de jonction pour représenter une alternative.

6

B. Point de décision :



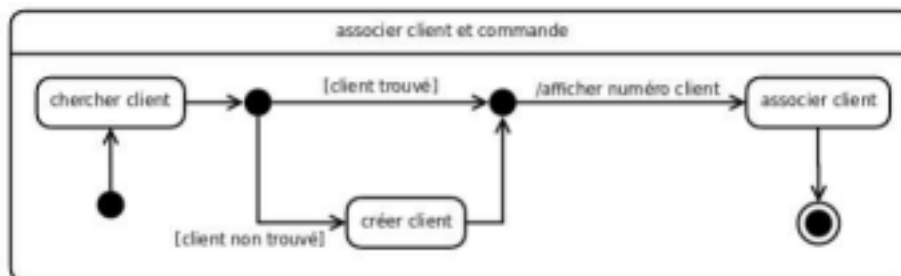
Exemple d'utilisation d'un point de décision.

3.4 Etat composite

Un état composite est un état qui peut être subdivisé en sous-états et qui peut contenir des transitions internes. Il est utilisé pour modéliser des comportements plus complexes et structurés au sein d'un état.

Des transitions peuvent être définies à l'intérieur de l'état composite, décrivant le comportement local sans quitter l'état composite.

L'état composite est une fonctionnalité essentielle dans les diagrammes d'états UML, offrant une manière structurée de modéliser des comportements complexes en les décomposant en sous-états et en utilisant des transitions



Exemple d'état composite modélisant l'association d'une commande à un client.

internes.

4. Diagramme d'activités

les diagrammes d'activité offrent une méthode visuelle et puissante pour modéliser le comportement dynamique des systèmes, facilitant la compréhension, la communication, et la conception des processus logiciels et métier.

Les diagrammes d'activité sont une extension des diagrammes d'états-transitions et sont utilisés pour modéliser graphiquement le déroulement des activités au sein d'un système. Ils offrent une représentation visuelle puissante des processus métier, des algorithmes et des flux de travail.

Ils sont donc particulièrement adaptés à la modélisation du cheminement de flots de contrôle et de flots de données. Ils permettent ainsi de représenter graphiquement le comportement d'une méthode ou le déroulement d'un cas d'utilisation.

Dans la phase de conception, les diagrammes d'activités sont particulièrement adaptés à la description des cas d'utilisation. Plus précisément, ils viennent illustrer et consolider la description textuelle des cas d'utilisation. De plus, leur représentation sous forme d'organigrammes les rend facilement intelligibles.

7

4.1 Eléments principaux

A- Action : Elle est le plus petit traitement qui puisse être exprimé en UML. Une action peut être, par exemple :

- une affectation de valeur à des attributs ;
- un accès à la valeur d'une propriété structurelle (attribut ou terminaison d'association) ; ■ la création d'un nouvel objet ou lien ;
- un calcul arithmétique simple ;
- l'émission d'un signal ;
- la réception d'un signal ;

B- Activité : Elle définit un comportement décrit par un séquençement organisé d'unités dont les éléments simples sont les actions. Le flot d'exécution est modélisé par des nœuds reliés par des arcs (transitions). Le flot de contrôle reste dans l'activité jusqu'à ce que les traitements soient terminés.

C- Nœud d'activité : Il modélise une action, permettant de représenter les étapes le long du flot d'une activité. Il existe trois familles de nœuds d'activités :

- Les nœuds d'actions : Ils représentent les éléments du processus qui effectuent des actions, qu'il s'agisse d'opérations simples ou complexes.
- Les nœuds objets : Ils représentent les instances d'objets ou d'entités manipulées dans le processus. Ils indiquent les entités sur lesquelles les actions sont effectuées. Si une action concerne une instance spécifique d'une classe, un nœud objet sera utilisé pour représenter cette instance.
- Nœuds de Contrôle : Ils sont utilisés pour représenter des éléments qui contrôlent le flux d'exécution dans le processus, tels que les décisions, les boucles, les fourches et les jonctions. Ils permettent donc de modéliser la logique conditionnelle, les boucles et d'autres structures de contrôle au sein du processus.

En résumé, les nœuds d'actions représentent les actions effectuées, les nœuds objets

représentent les entités impliquées, et les nœuds de contrôle modélisent les structures de contrôle. Ensemble, ces éléments permettent de créer des diagrammes d'activité complets et compréhensibles, illustrant le déroulement du processus de manière claire et visuelle.

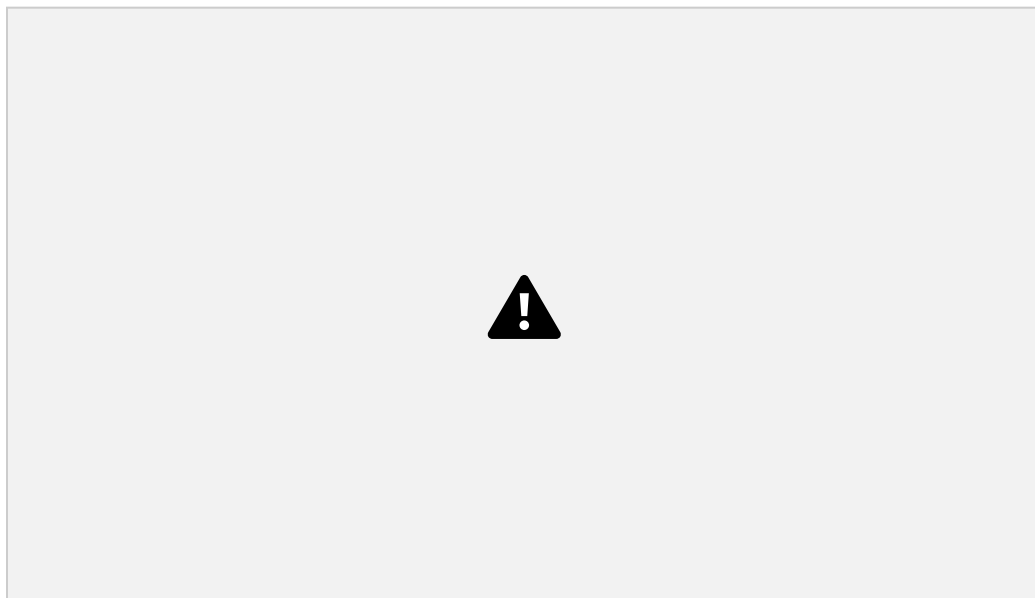
La figure suivante représente le nœud représentant une action, un nœud objet, un nœud de décision ou de fusion, un nœud de bifurcation ou d'union, un nœud initial, un nœud final et un nœud final de flot.

D- Une transition : Elle représente le passage d'un état ou d'une activité à un autre. Les transitions sont utilisées pour décrire les changements de statut ou de contexte au sein d'un processus ou d'un workflow. Les transitions sont symbolisées par des flèches reliant les nœuds d'activités, montrant la direction du flux. Elles sont généralement étiquetées pour indiquer la condition ou l'événement déclencheur de la transition.

8

4.2 Exemple illustratif

Soit le diagramme d'activité qui décrit la prise en compte d'une commande. Ce diagramme illustre l'utilisation de nœuds de contrôle.



9

5. Diagramme de collaboration

Les diagrammes de collaboration, également appelés diagrammes de communication en UML, sont utilisés pour modéliser les interactions et les collaborations entre les objets au sein d'un système logiciel. Cette représentation se concentre sur les relations d'interaction entre les objets. La dimension temporelle est ajoutée grâce à des numéros de

séquence.

Un diagramme de collaboration représente un ensemble de rôles joués par les objets dans un contexte particulier, ainsi que les liens entre ces objets. Le passage à un diagramme de séquence et inversement est simple.

5.1 Éléments principaux

A- Collaboration : Elle modélise une activité réalisée par un ensemble d'objets. Elle peut représenter un scénario d'un cas d'utilisation du point de vue du fonctionnement interne du système.

B- Objets : Ils représentent les instances des classes participant à la collaboration. Chaque objet est généralement représenté par un rectangle contenant le nom de l'objet.

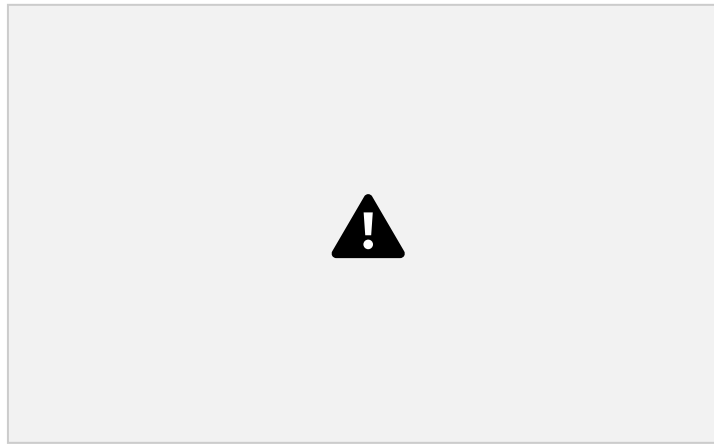
C- Messages : Ils représentent les communications entre les objets. Les messages sont généralement ordonnés selon un numéro de séquence croissant.

D- Associations et Liens : Des lignes et des flèches peuvent être utilisées pour représenter des associations et des liens entre les objets.

E- Acteur : La notation UML permet de faire figurer un acteur dans les diagrammes de collaboration. Ils permettent de représenter les interactions déclenchées par un élément extérieur au système.



Exemple :



Un diagramme de collaboration décrit le comportement du système pour chacun des scénarios accompagnant les cas d'utilisation.

Remarque : Si les opérations ont une structure algorithmique, on préférera les décrire avec des diagrammes d'activités. Les diagrammes de collaboration sont plus adaptés quand une opération fait interagir de nombreux objets.

5.2 Différence entre diagramme de séquence et de collaboration

Les diagrammes de séquence et les diagrammes de collaboration sont deux types de diagrammes UML qui se concentrent sur la modélisation des interactions entre les objets, mais ils le font de manière différente. Le diagramme de séquence met l'accent sur la chronologie des interactions entre les objets au fil du temps. Il montre comment les messages sont échangés entre les objets au cours de l'exécution. Par contre, le diagramme de collaboration met l'accent sur la visualisation des interactions entre les objets, mais sans se préoccuper de la chronologie spécifique des messages. Les messages sont souvent représentés de manière plus informelle.

En résumé, le diagramme de séquence se concentre davantage sur la séquence temporelle des interactions entre les objets, tandis que le diagramme de collaboration met l'accent sur la visualisation des relations structurelles et des interactions entre les objets sans se soucier de la séquence temporelle précise des messages. Le choix entre ces deux types de diagrammes dépend des objectifs de modélisation spécifiques et du niveau de détail requis. Souvent, ces deux types de diagrammes sont utilisés ensemble pour fournir une vue complète des interactions dans un système logiciel.

